

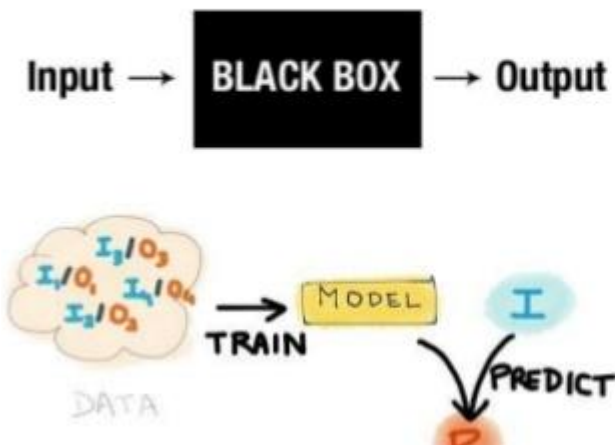
CSE 445: Machine Learning

Training Models



Course so far

- Treated ML algorithms as black boxes (mostly)
- Learned how to preprocess data, and how to evaluate ML models
- Time to look under the hood!
 - Linear Regression
 - Gradient Descent
 - Polynomial Regression
 - Logistic Regression
 - Softmax Regression



Linear Regression

- Two ways to train an LR model
1. Direct “closed-form” equation for computing model parameters
 - Yields the model parameters θ that provides the best fit directly – i.e. the parameters that minimize the cost function
 2. Iterative optimization via Gradient Descent, or one of its variants (Batch GD, Mini-batch GD, Stochastic GD)
 - Gradually tweak the model parameters to minimize cost function of training set
 - Ideally will converge to the same set of parameters as the direct method

Linear Regression

- Make prediction by computing a **weighted sum** of input **features** and a constant **bias term**

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

- $\hat{y} \rightarrow$ predicted value - scalar
- $n \rightarrow$ number of features
- $x_i \rightarrow$ i^{th} feature value
- $\theta_j \rightarrow$ j^{th} model parameter
- Find the vector θ that minimizes the cost function (**Mean Squared Error**)

$$\text{MSE}(\mathbf{X}, \mathbf{h}_{\theta}) = \frac{1}{m} \sum_{i=1}^m (\theta^T \mathbf{x}^{(i)} - y^{(i)})^2$$

- Can write this more concisely in **vectorized** form

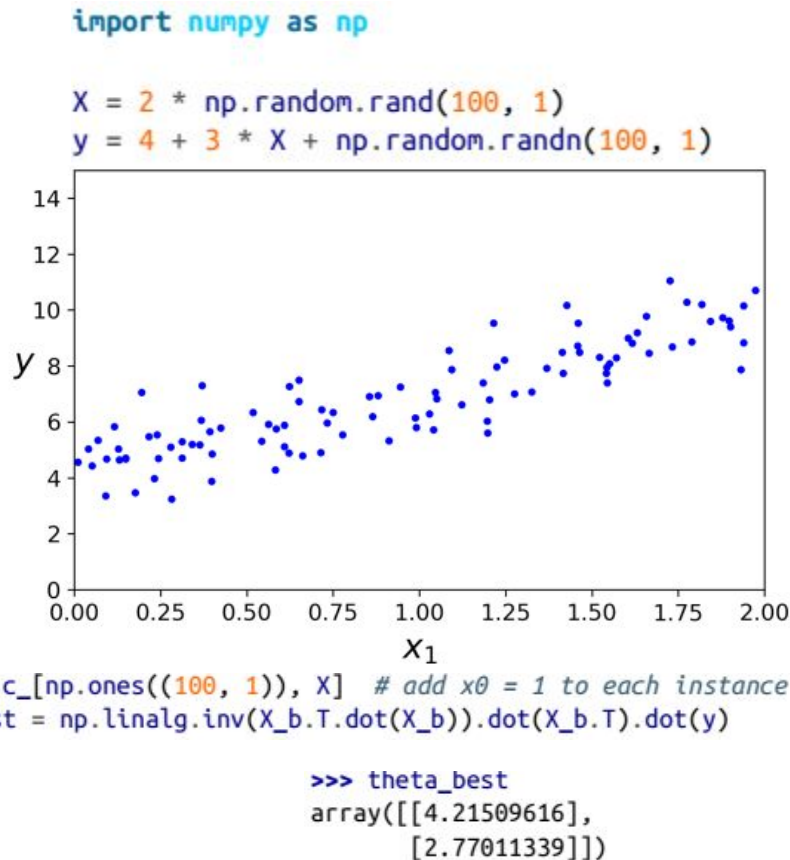
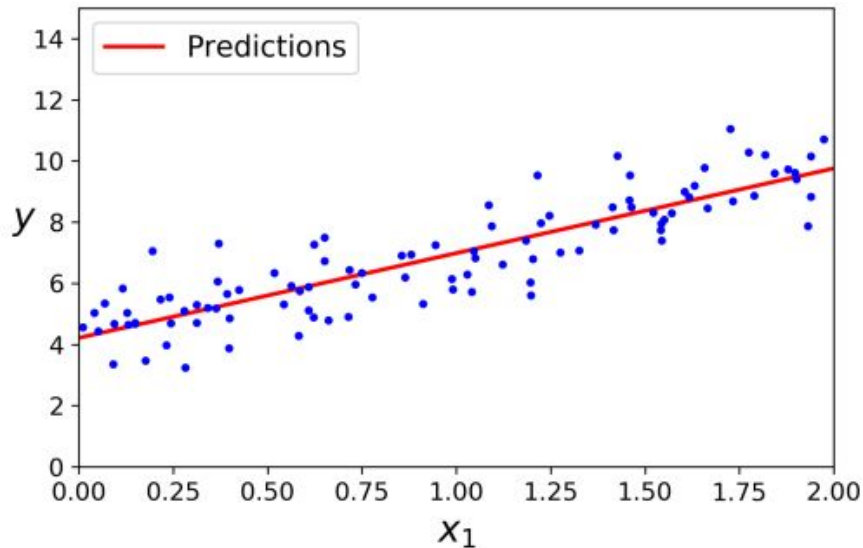
$$\hat{y} = h_{\theta}(\mathbf{x}) = \theta \cdot \mathbf{x}$$

- $\theta \rightarrow$ parameter vector, containing bias term θ_0 and feature weights θ_1 to θ_n
- $\mathbf{x} \rightarrow$ feature vector of the instance from x_0 to x_n [$x_0 = 1$]
- $\theta \cdot \mathbf{x} \rightarrow$ dot product of θ and $\mathbf{x}$
- $h_{\theta} \rightarrow$ hypothesis function
- Note: if both θ and $\mathbf{x}$ are column vectors, then $\hat{y} = \theta^T \mathbf{x}$ (T superscript means

Closed Form: Normal Equation

- Closed form solution for the value of θ that minimizes the cost function

$$\hat{\theta} = (X^T X)^{-1} X^T y$$



Closed Form: SVD Approach

- Can compute theta by multiplying the pseudoinverse (Moore-Penrose inverse) to the target vector

$$\hat{\theta} = \mathbf{X}^+ \mathbf{y}$$

- Pseudoinverse $\mathbf{X}^+$ is computed by performing Singular Value Decomposition (SVD)
 - $\mathbf{X}$ is broken into the matrix multiplication of 3 matrices $\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$
 - $\mathbf{X}^+$ computed as $\mathbf{X}^+ = \mathbf{V}\mathbf{\Sigma}^+ \mathbf{U}^T$
 - $\mathbf{\Sigma}^+$ computed by setting all values in $\mathbf{\Sigma}$ below a threshold to zero, and inverting the nonzero values, and transposing the resulting matrix

```
>>> from sklearn.linear_model import LinearRegression
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([4.21509616]), array([[2.77011339]]))
>>> lin_reg.predict(X_new)
array([[4.21509616],
       [9.75532293]])
```

```
>>> theta_best_svd, residuals, rank, s = np.linalg.lstsq(X_b, y, rcond=1e-6)
>>> theta_best_svd
array([[4.21509616],
       [2.77011339]])
```

```
>>> np.linalg.pinv(X_b).dot(y)
array([[4.21509616],
       [2.77011339]])
```

Normal Equation

- Computational Complexity of finding $(\mathbf{X}^T \mathbf{X})^{-1}$ is $\mathbf{O}(n^{2.4})$ to $\mathbf{O}(n^3)$ depending on implementation
 - Doubling number of features n causes computation time to go up by 5.3 – 8 times!
- Normal Equation may not work if the matrix $\mathbf{X}^T \mathbf{X}$ is not invertible (i.e. singular)
 - If $m < n$ [m : number of instances, n : number of features]
 - If some features are redundant
- SVD approach is $\sim \mathbf{O}(n^2)$ - more efficient than Normal Equation
 - Pseudoinverse is always defined
 - Doubling number of features multiplies computation time by ~ 4
- Both approaches can be difficult to train if there are a large number of features, or if there are too many training instances to fit in memory.

Gradient Descent

■ Optimization algorithm

- tweak parameters iteratively to minimize a cost function
- Measure local gradient of the error function wrt the parameter vector θ , and go in the direction of descending gradient
 - Moving downhill in the direction of steepest slope
 - Once gradient reaches zero, you've reached a minimum
 - Initialize θ with random values. and

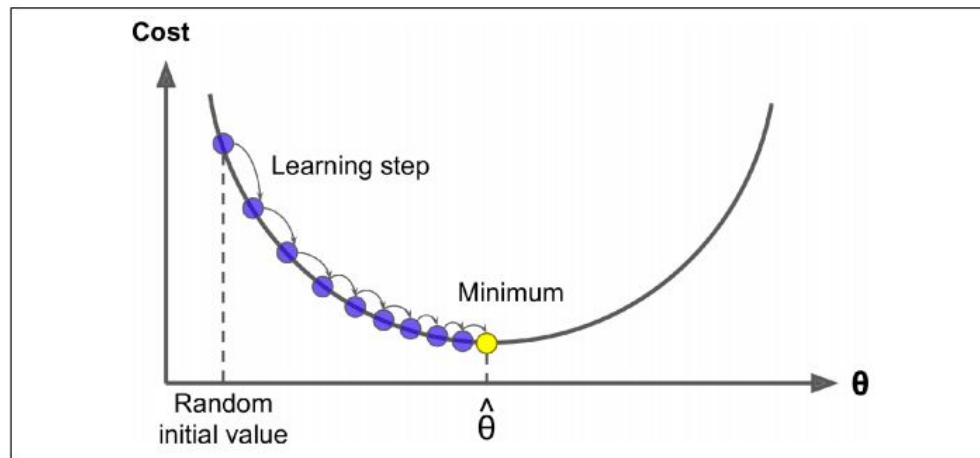


Figure 4-3. In this depiction of Gradient Descent, the model parameters are initialized randomly and get tweaked repeatedly to minimize the cost function; the learning step size is proportional to the slope of the cost function, so the steps gradually get smaller as the parameters approach the minimum

Gradient Descent – Low Learning Rate

- Size of the steps determined by the learning rate hyperparameter
- If learning rate is too low, the algorithm needs far too many iterations to converge

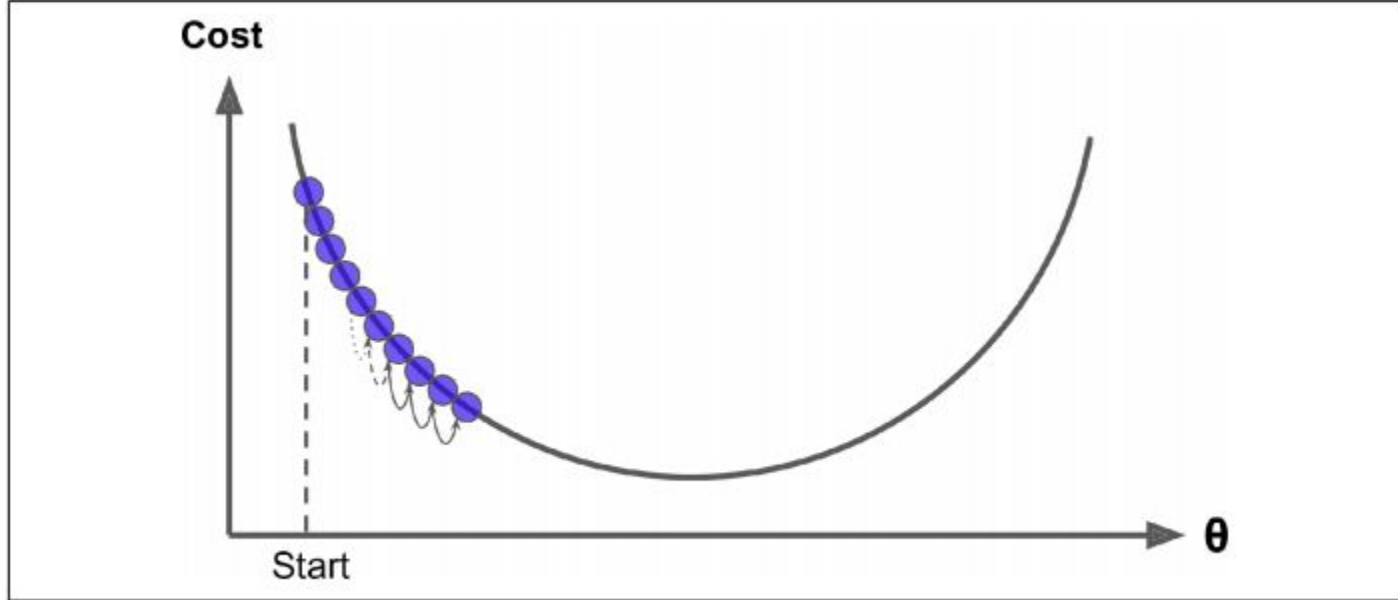


Figure 4-4. The learning rate is too small

Gradient Descent – High Learning Rate

- If learning rate is too high, you might jump across the valley and end up on other side □ might make the algorithm diverge!

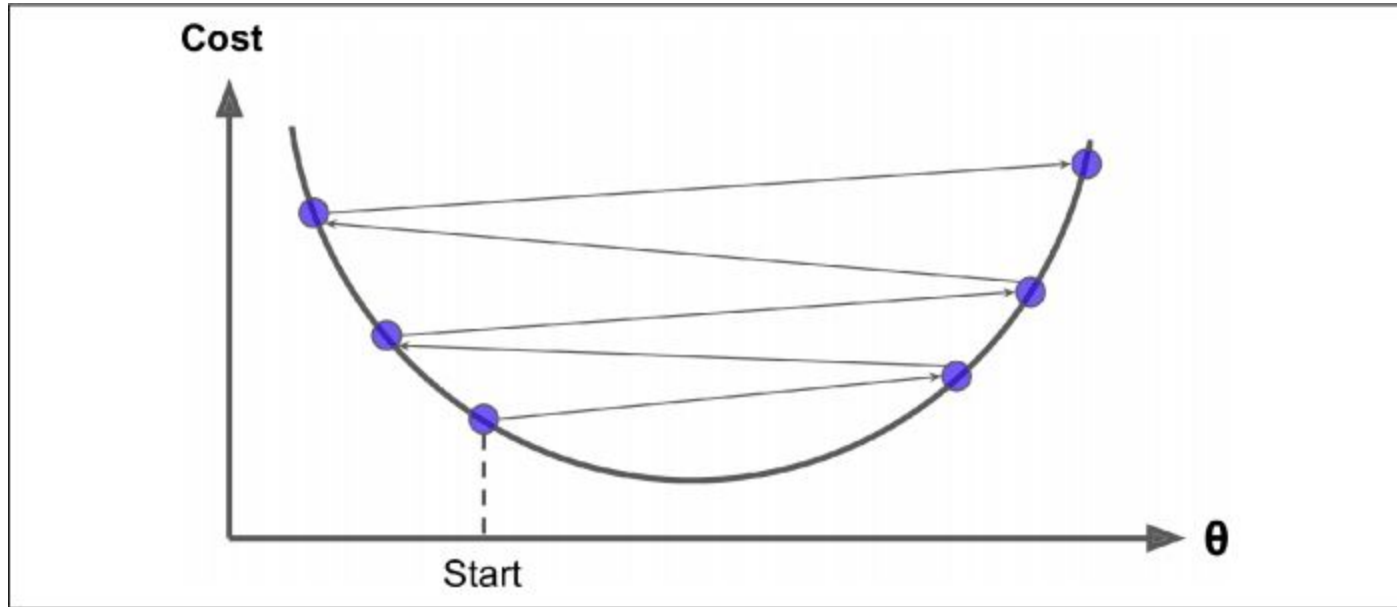


Figure 4-5. The learning rate is too large

Gradient Descent – Nonconvex cost function

- Cost function may not be a nice bowl with a single minimum - could have holes, ridges, plateaus etc
 - MSE cost function happens to be a convex function ☐ pick any two points on the curve, and a straight line joining them will never intersect the curve

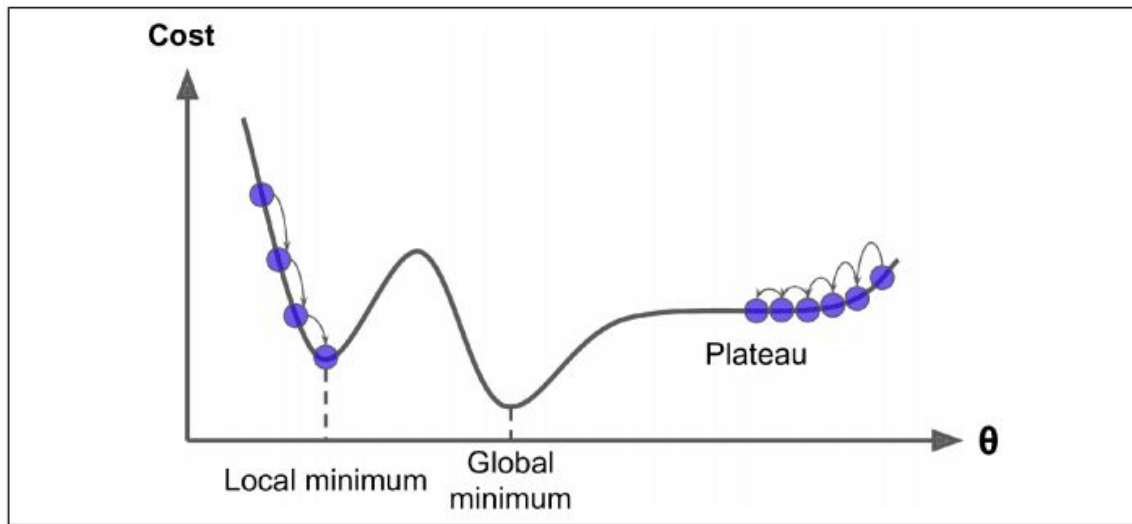


Figure 4-6. Gradient Descent pitfalls

Gradient Descent – Feature Scaling

- If features are not scaled, Gradient descent may take a long time to approach the global minimum
- The more parameters in model $\square$ more dimensions in parameter space $\square$ harder to find global minimum

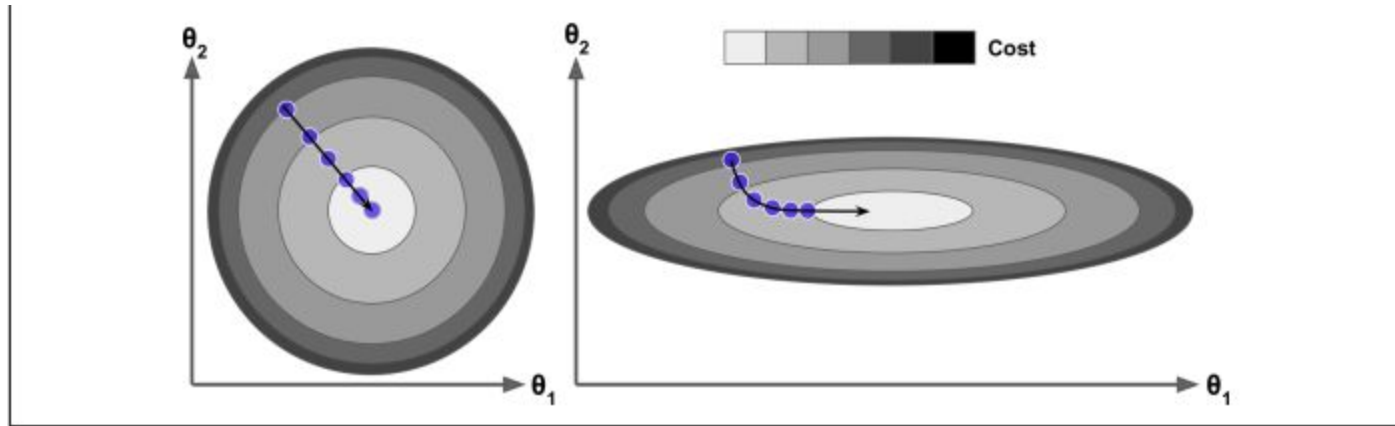


Figure 4-7. Gradient Descent with (left) and without (right) feature scaling

Batch Gradient Descent

- Find the partial derivative $\frac{\partial}{\partial \theta_j}$ of cost function regarding each individual parameter
- Can vectorize and compute all partial derivatives in one go - $\nabla_{\theta} MSE(\theta)$ contains all PDs of the cost function
- Gradient vector points uphill – go in the opposite direction to find the minimum, by subtracting $\nabla_{\theta} MSE(\theta)$ and the learning rate η
- Calculations done over full training set! But still faster than SVD or normal

$$\frac{\partial}{\partial \theta_j} MSE(\theta) = \frac{2}{m} \sum_{i=1}^m (\theta^T \mathbf{x}^{(i)} - y^{(i)}) x_j^{(i)}$$

$$\nabla_{\theta} MSE(\theta) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} MSE(\theta) \\ \frac{\partial}{\partial \theta_1} MSE(\theta) \\ \dots \\ \frac{\partial}{\partial \theta_n} MSE(\theta) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y})$$

$$\theta = \theta - \eta \nabla_{\theta} MSE(\theta)$$

Learning Rate

- Learning rate can affect speed of convergence significantly
- Stop training when gradient vector becomes very tiny – when its norm is smaller than ϵ (tolerance)

```
eta = 0.1 # learning rate
n_iterations = 1000
m = 100

theta = np.random.randn(2,1) # random initialization

for iteration in range(n_iterations):
    gradients = 2/m * X_b.T.dot(X_b.dot(theta) - y)
    theta = theta - eta * gradients

>>> theta
array([[4.21509616],
       [2.77011339]])
```

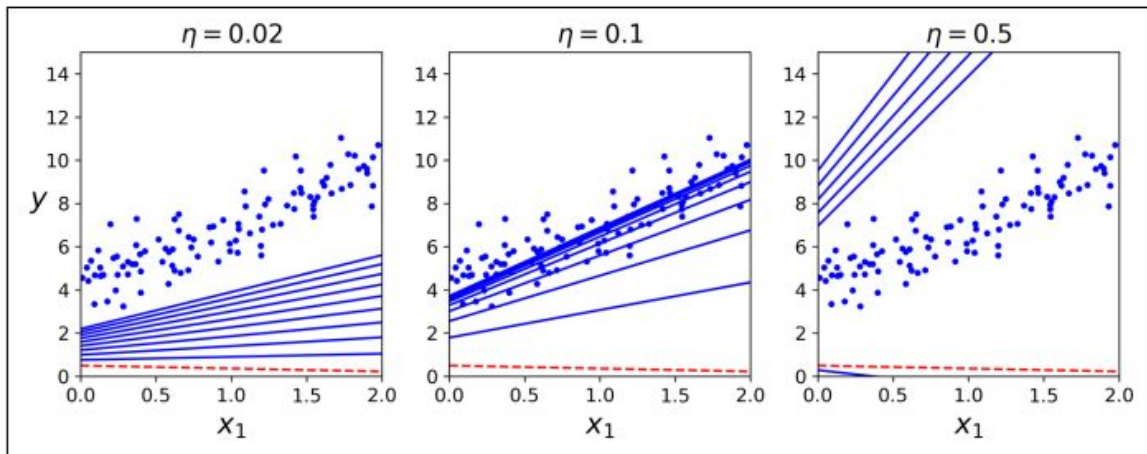


Figure 4-8. Gradient Descent with various learning rates

Stochastic Gradient Descent

- Batch GD with full training set □ very slow with large training sets!
- Stochastic GD □ random instance in training set picked at every step for gradient compute (only one instance in memory per iteration)
- Only decreases on average – will bounce around optimal value

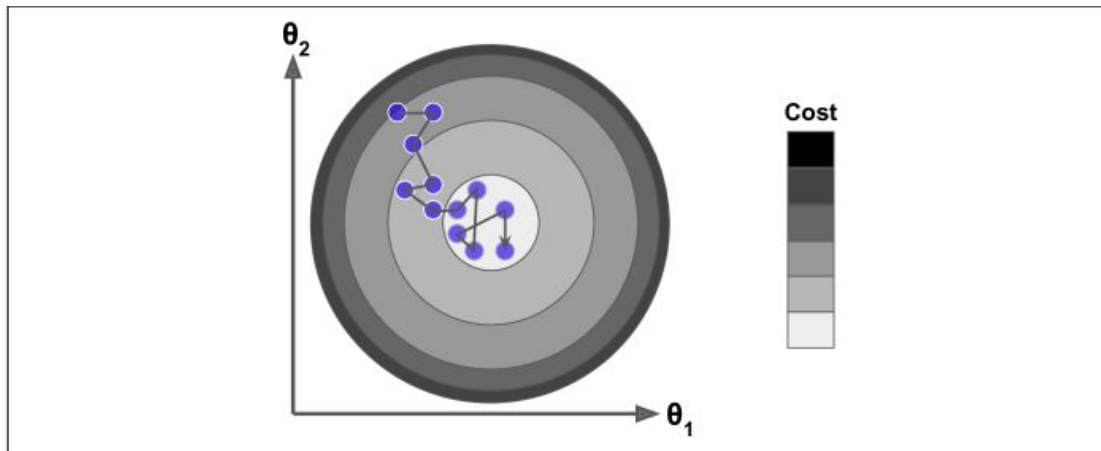


Figure 4-9. With Stochastic Gradient Descent, each training step is much faster but also much more stochastic than when using Batch Gradient Descent

- Good when cost function is very irregular
- Reduce learning rate to help settle on minimum

Stochastic Gradient Descent

- Learning schedule □ gradually lower learning rate
- Epoch □ each round of m iterations
 - Just 50 epochs total, instead of 1000 iterations through whole training set
 - Shuffling required to ensure instances are independent and identically distributed

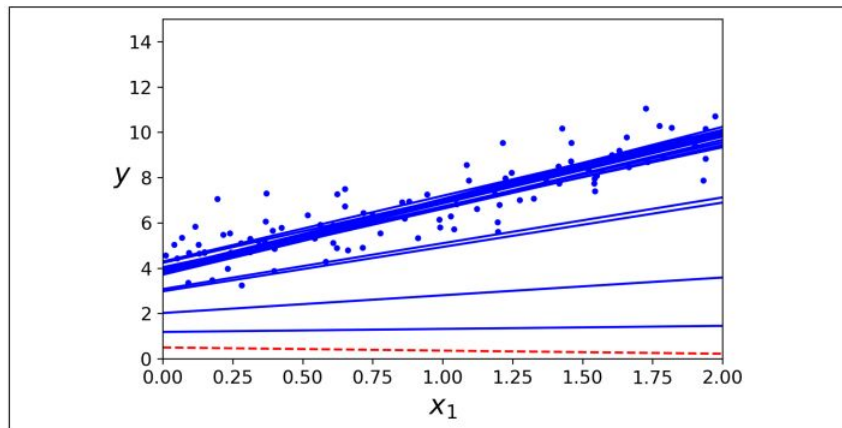


Figure 4-10. The first 20 steps of Stochastic Gradient Descent

```
n_epochs = 50
t0, t1 = 5, 50 # learning schedule hyperparameters

def learning_schedule(t):
    return t0 / (t + t1)

theta = np.random.randn(2,1) # random initialization

for epoch in range(n_epochs):
    for i in range(m):
        random_index = np.random.randint(m)
        xi = X_b[random_index:random_index+1]
        yi = y[random_index:random_index+1]
        gradients = 2 * xi.T.dot(xi.dot(theta) - yi)
        eta = learning_schedule(epoch * m + i)
        theta = theta - eta * gradients

>>> theta
array([[4.21076011],
       [2.74856079]])

from sklearn.linear_model import SGDRegressor
sgd_reg = SGDRegressor(max_iter=1000, tol=1e-3, penalty=None, eta0=0.1)
sgd_reg.fit(X, y.ravel())

>>> sgd_reg.intercept_, sgd_reg.coef_
(array([4.24365286]), array([2.8250878]))
```


Mini-batch Gradient Descent

- Compute gradients on small, random sets of instances called mini-batches.
- Best of both worlds – computationally less expensive than Batch GD, and less erratic than Stochastic GD
- Good learning schedule required to get to global minimum

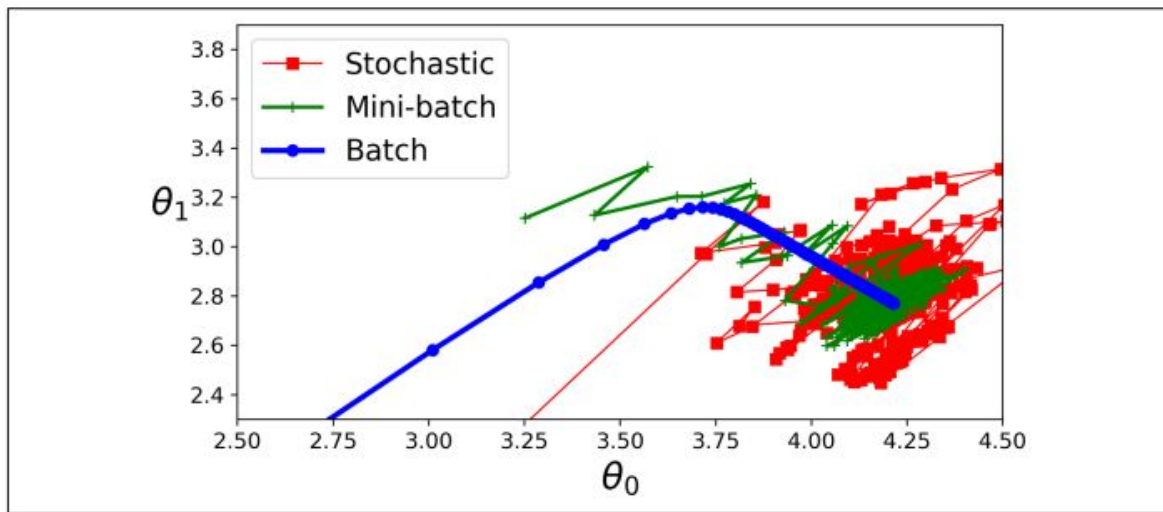


Figure 4-11. Gradient Descent paths in parameter space

Table 4-1. Comparison of algorithms for Linear Regression

Algorithm	Large m	Out-of-core support	Large n	Hyperparams	Scaling required	Scikit-Learn
Normal Equation	Fast	No	Slow	0	No	N/A
SVD	Fast	No	Slow	0	No	LinearRegression
Batch GD	Slow	No	Fast	2	Yes	SGDRegressor
Stochastic GD	Fast	Yes	Fast	≥ 2	Yes	SGDRegressor
Mini-batch GD	Fast	Yes	Fast	≥ 2	Yes	SGDRegressor

Polynomial Regression

- Append powers of each feature as new features
- Train a linear model on the extended set of features
- Capable of finding relationships between features as well – not possible with just linear regression

```
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X**2 + X + 2 + np.random.randn(m, 1)

>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly_features = PolynomialFeatures(degree=2, include_bias=False)
>>> X_poly = poly_features.fit_transform(X)
>>> X[0]
array([-0.75275929])
>>> X_poly[0]
array([-0.75275929, 0.56664654])

>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X_poly, y)
>>> lin_reg.intercept_, lin_reg.coef_
(array([1.78134581]), array([[0.93366893, 0.56456263]]))
```

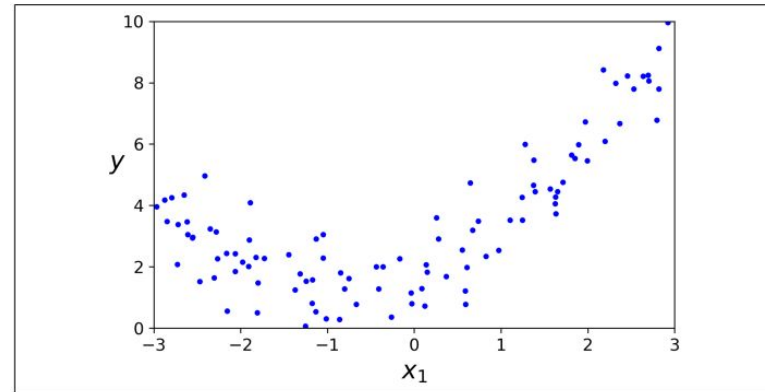


Figure 4-12. Generated nonlinear and noisy dataset

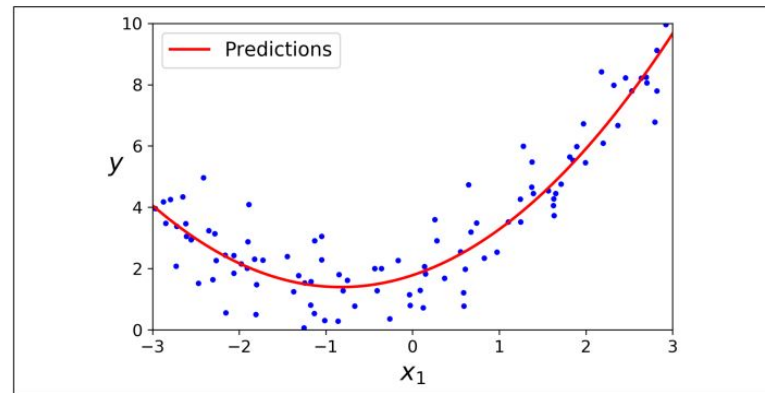


Figure 4-13. Polynomial Regression model predictions

Is your model overfitting or underfitting?

- High degree polynomial model ☐ severe overfit
- Linear model ☐ severe underfit
- One way to know is via cross validation
 - If model does well on training, but poorly on validation, it's overfitting
 - If it's poor on both, it's underfitting
- Alternative way is via Learning Curves
 - plots of model performance on training and validation set as a function of training set size or training iteration

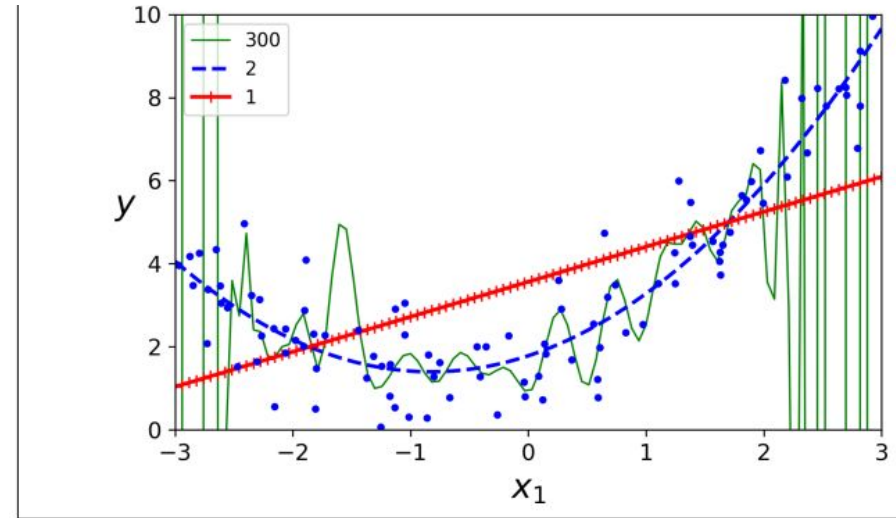


Figure 4-14. High-degree Polynomial Regression

Learning Curves - Underfitting

- Linear regression model fitted on quadratic data
- Few instances $\square$ can be fit perfectly, so low RMSE when training set size is little
- Error in training data goes up and plateaus – adding new instances doesn't make the error change
- Initial validation error large $\square$ model cannot generalize properly
- Validation curve also plateaus and stays close to the training curve

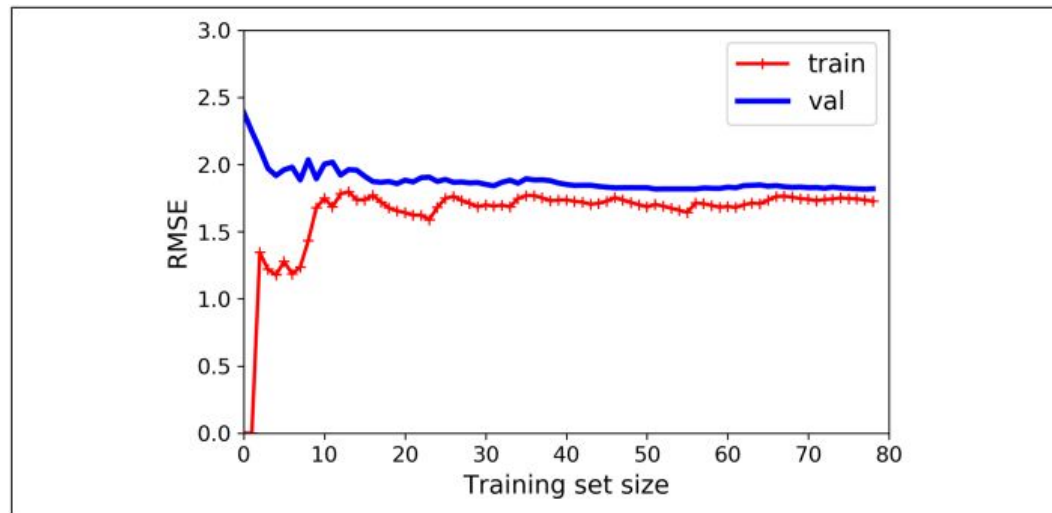


Figure 4-15. Learning curves

Learning Curves - Overfitting

- 10th degree polynomial model trained on quadratic data
- Error on training data is much lower
- Significant gap between training and validation curves □ i.e. the model performs much better on the training set than on the validation set
- Can be improved by feeding more training data until the two curves start to get closer

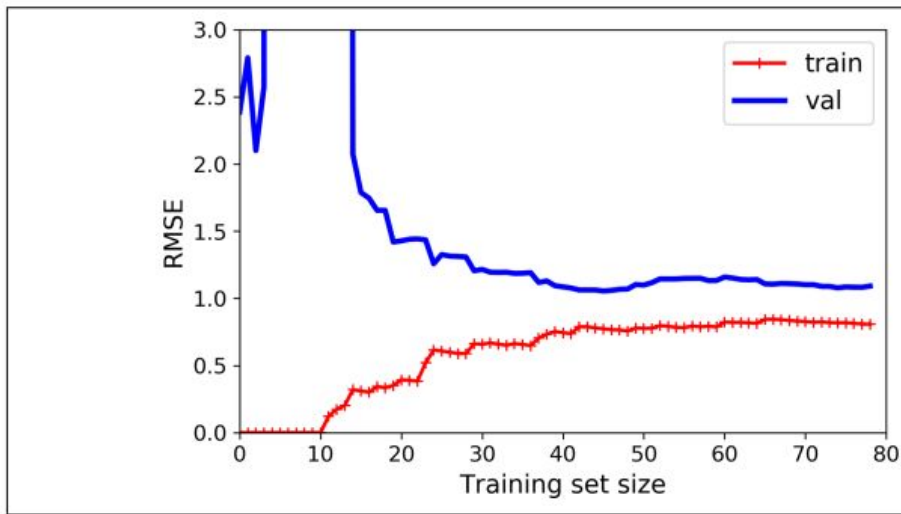


Figure 4-16. Learning curves for the 10th-degree polynomial model

Bias/Variance Trade-off

ML model's generalization error comes from three sources:

1. Bias $\square$ generalization error due to wrong assumptions
 - e.g. assuming the data is linear when it's actually quadratic
 - High bias will lead to underfitting
2. Variance $\square$ high sensitivity to small variations in training data
 - e.g. polynomial model with many degrees of freedom on quadratic data
 - High variance leads to overfitting
3. Irreducible error $\square$ due to inherent noise of the data itself
 - Make models more complex $\square$ higher variance, lower bias, overfitting
 - Make models less complex $\square$ lower variance, higher bias, underfitting
 - Need to strike a very delicate balance between the two

Ridge Regression

- Also called Tikhonov regularization
- Regularized version of linear regression – a regularization term is added to the cost function
- Algorithm has to keep model weights as small as possible during training
- Half of L2 norm used to limit the weight vector

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \alpha \frac{1}{2} \sum_{i=1}^n \theta_i^2$$

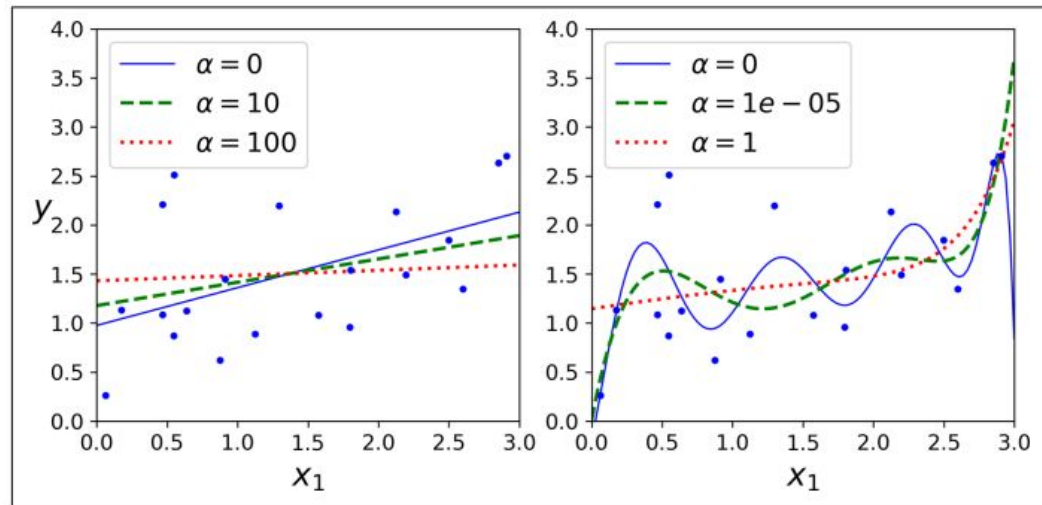


Figure 4-17. A linear model (left) and a polynomial model (right), both with various levels of Ridge regularization

```
>>> sgd_reg = SGDRegressor(penalty="l2")  
>>> sgd_reg.fit(X, y.ravel())  
>>> sgd_reg.predict([[1.5]])  
array([1.47012588])
```

Lasso Regression

- Least Absolute Shrinkage and Selection Operator(LASSO) regression
- L1 norm of weight vector used to regularize
- Eliminates the weights of least important features
- Automatic feature selection and sparse model output

$$J(\boldsymbol{\theta}) = MSE(\boldsymbol{\theta}) + \alpha \sum_{i=1}^n |\theta_i|$$

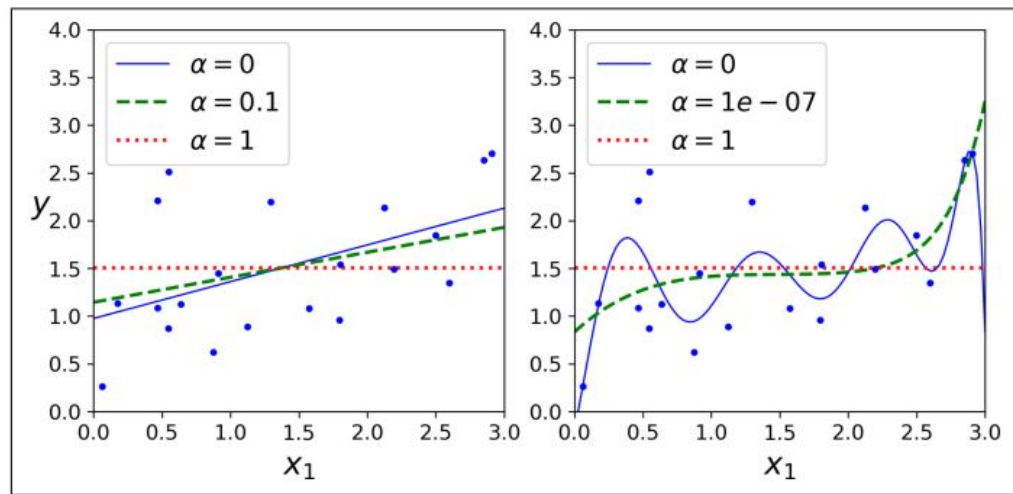


Figure 4-18. A linear model (left) and a polynomial model (right), both using various levels of Lasso regularization

Lasso vs Ridge

- Contours $\rightarrow$ loss (left), cost function (right)
- Yellow line $\rightarrow$ initialized to $\theta_1 = 2$ and $\theta_2 = 0.5$
- White line $\rightarrow$ initialized to $\theta_1 = 0.25$ and $\theta_2 = -1$
- Global optimum (red square) not reached with Lasso as θ_2 gets set to zero
 - Gradually reduce learning rate α during training to avoid GD from bouncing around optimum in Lasso
 - Elastic net $\rightarrow$ middle ground between

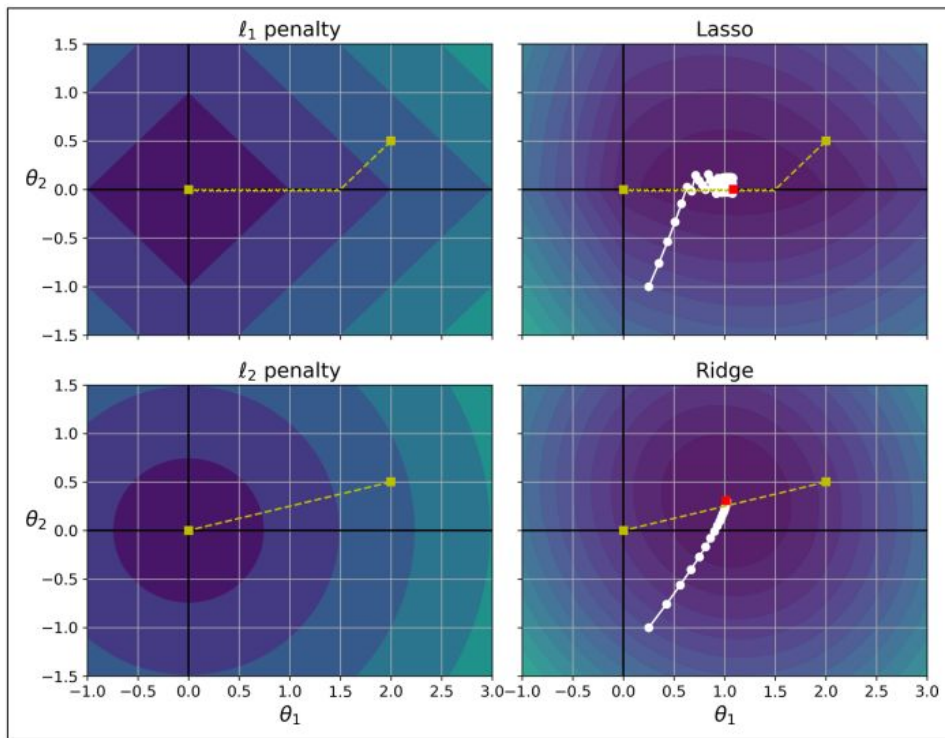


Figure 4-19. Lasso versus Ridge regularization

Early Stopping

- Stop training as soon as validation error reaches a minimum
 - When validation RMSE begins to go up whilst training RMSE is still going down, that indicates overfitting

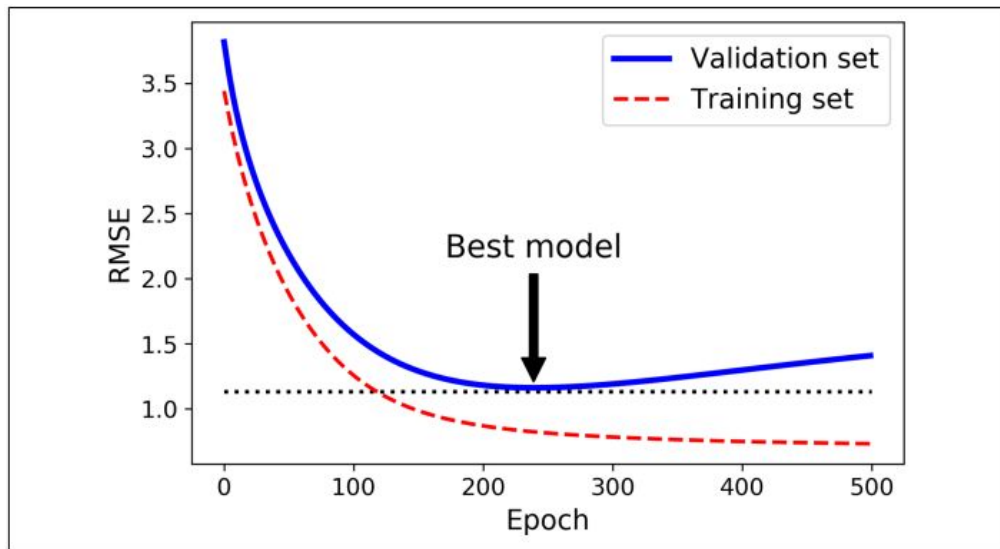


Figure 4-20. Early stopping regularization

```
from sklearn.base import clone

# prepare the data
poly_scaler = Pipeline([
    ("poly_features", PolynomialFeatures(degree=90, include_bias=False)),
    ("std_scaler", StandardScaler())
])
X_train_poly_scaled = poly_scaler.fit_transform(X_train)
X_val_poly_scaled = poly_scaler.transform(X_val)

sgd_reg = SGDRegressor(max_iter=1, tol=-np.infty, warm_start=True,
                       penalty=None, learning_rate="constant", eta0=0.0005)

minimum_val_error = float("inf")
best_epoch = None
best_model = None
for epoch in range(1000):
    sgd_reg.fit(X_train_poly_scaled, y_train) # continues where it left off
    y_val_predict = sgd_reg.predict(X_val_poly_scaled)
    val_error = mean_squared_error(y_val, y_val_predict)
    if val_error < minimum_val_error:
        minimum_val_error = val_error
        best_epoch = epoch
        best_model = clone(sgd_reg)
```

Logistic Regression

- Binary classifier- also called Logit Regression, but used in classification tasks
- The probability that an instance belongs to a particular class is estimated
- If estimated probability is more than 50%, the model predicts the instance to belong to that class – otherwise, it predicts it does not belong to that class

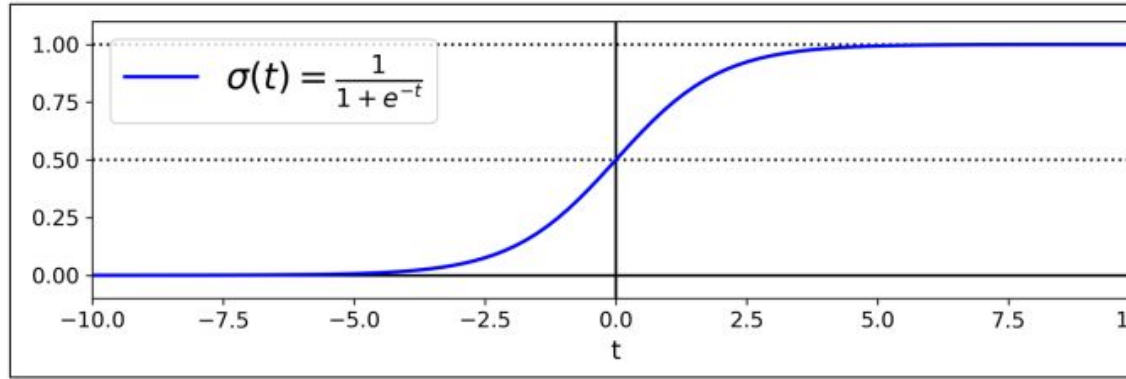


Figure 4-21. Logistic function

- Estimated probability:
$$\hat{p} = h_{\theta}(\mathbf{x}) = \sigma(\mathbf{x}^T \boldsymbol{\theta})$$
 - $\sigma()$ is the sigmoid function that outputs a number between 0 and 1
- $$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5 \\ 1 & \text{if } \hat{p} \geq 0.5 \end{cases}$$

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

Logistic Regression Training

- Model estimates high probabilities for positive instances ($y=1$) and low probabilities for negative instances ($y=0$)
- Cost Function for single training instance:

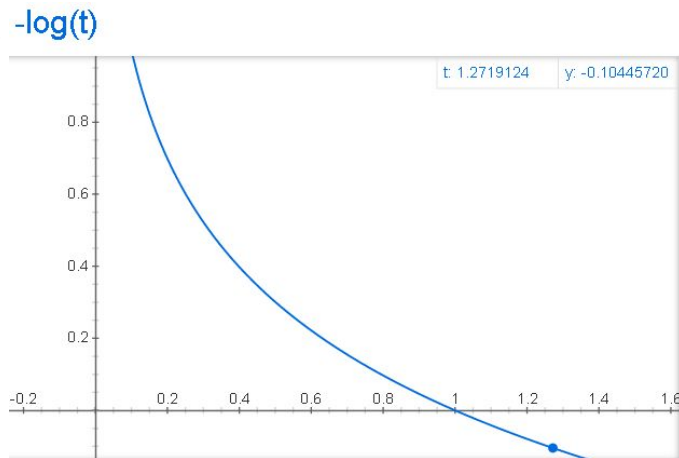
$$c(\theta) = \begin{cases} -\log(\hat{p}) & \text{if } y = 1 \\ -\log(1 - \hat{p}) & \text{if } y = 0 \end{cases}$$

- Log Loss $\rightarrow$ average cost over all training instances

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)})]$$

- The cost function is convex, so GD guaranteed to find global minimum

$$\frac{\partial}{\partial \theta} \frac{1}{m} \sum_{i=1}^m$$



Logistic Regression example – Iris flower classifier

- Classify iris flowers of three different species



Figure 4-22. Flowers of three iris plant species¹⁴

Logistic Regression example

```
>>> from sklearn import datasets
>>> iris = datasets.load_iris()
>>> list(iris.keys())
['data', 'target', 'target_names', 'DESCR', 'feature_names', 'filename']
>>> X = iris["data"][:, 3:] # petal width
>>> y = (iris["target"] == 2).astype(np.int) # 1 if Iris virginica, else 0
from sklearn.linear_model import LogisticRegression
```

```
log_reg = LogisticRegression()
log_reg.fit(X, y)
```

```
X_new = np.linspace(0, 3, 1000).reshape(-1, 1)
y_proba = log_reg.predict_proba(X_new)
plt.plot(X_new, y_proba[:, 1], "g-", label="Iris virginica")
plt.plot(X_new, y_proba[:, 0], "b--", label="Not Iris virginica")
# + more Matplotlib code to make the image look pretty
```

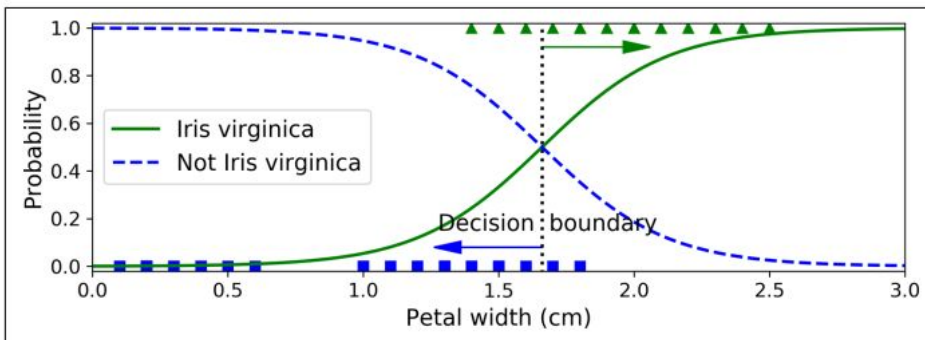
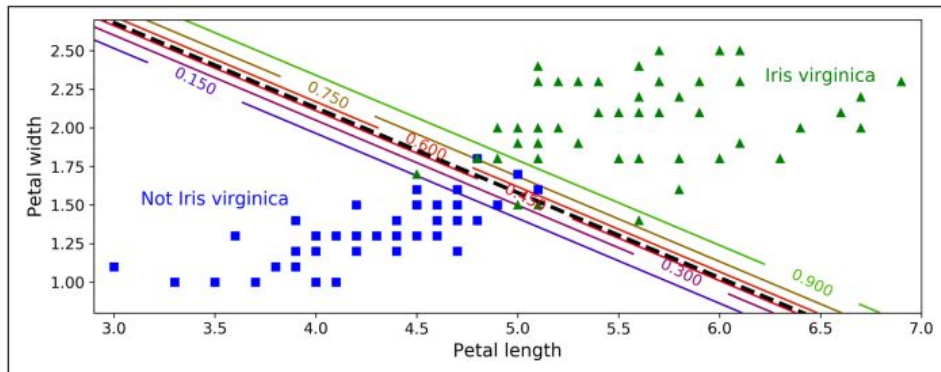


Figure 4-23. Estimated probabilities and decision boundary

- Binary classifier for iris virginica flowers
- Overlap between 1.4cm and 1.8cm width
- Decision boundary at 1.6cm
- Trained on two features ☐ can see linear boundary where model estimates 50% probability



Softmax Regression

- Also called Multiclass Logistic Regression/ Multinomial Logistic Regression

- Generalization of Logistic Regression to support multiple classes directly

- Compute a score $s_k(\mathbf{x})$ for each class k
 $s_k(\mathbf{x}) = \mathbf{x}^T \boldsymbol{\theta}^{(k)}$

- Estimate probability of each class by applying softmax function to the scores – computes the exponential of every score, and normalizes them

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}$$

- $K \rightarrow$ number of classes
- $\mathbf{s}(\mathbf{x}) \rightarrow$ vector containing scores for each class for instance $\mathbf{x}$
- $\sigma(\mathbf{s}(\mathbf{x}))_k \rightarrow$ estimated probability that instance $\mathbf{x}$ belongs to class k , given the scores

- Predicts the class with the highest estimated probability (like logistic regression)

$$\begin{aligned}\hat{y} &= \operatorname{argmax}_k \sigma(\mathbf{s}(\mathbf{x}))_k \\ &= \operatorname{argmax}_k s_k(\mathbf{x}) \\ &= \operatorname{argmax}_k ((\boldsymbol{\theta}^{(k)})^T \mathbf{x})\end{aligned}$$

- Note: argmax returns the value of k that maximizes the estimated probability

Softmax Regression – Cross Entropy

- Objective is to have model that estimates high prob. for target class, and low for the others
- Cost Function used is called cross entropy – penalizes the model when it estimates low prob. for target class

$$J(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)})$$

- $y_k^{(i)}$ is the target probability that i^{th} instance belongs to class k (either 1 or 0), and $\boldsymbol{\theta}$ is the parameter matrix
- For a case with two classes only, this function

- Gradient vector of this cost function wrt $\boldsymbol{\theta}^{(k)}$ is given as follows:

$$\nabla_{\boldsymbol{\theta}^{(k)}} J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m (\hat{p}_k^{(i)} - y_k^{(i)}) \mathbf{x}^{(i)}$$

- Can use this to compute gradient vectors for each class, and use gradient descent (or any other optimizer) to find parameter matrix $\boldsymbol{\theta}$ that minimizes cost function

Softmax Regression – Iris Petals

- Can choose Softmax Regression by setting `multi_class="multinomial"`
- L-BGFS optimizer
- L2 regularization by default
- Q: which flower are we looking at if:

- A) width = 2cm, length=5cm
- B) width = 1cm, length=1cm
- C) width = 3cm, length=2.5cm

```
X = iris["data"][:, (2, 3)] # petal length, petal width  
y = iris["target"]
```

```
softmax_reg = LogisticRegression(multi_class="multinomial", solver="lbfgs", C=10)  
softmax_reg.fit(X, y)
```

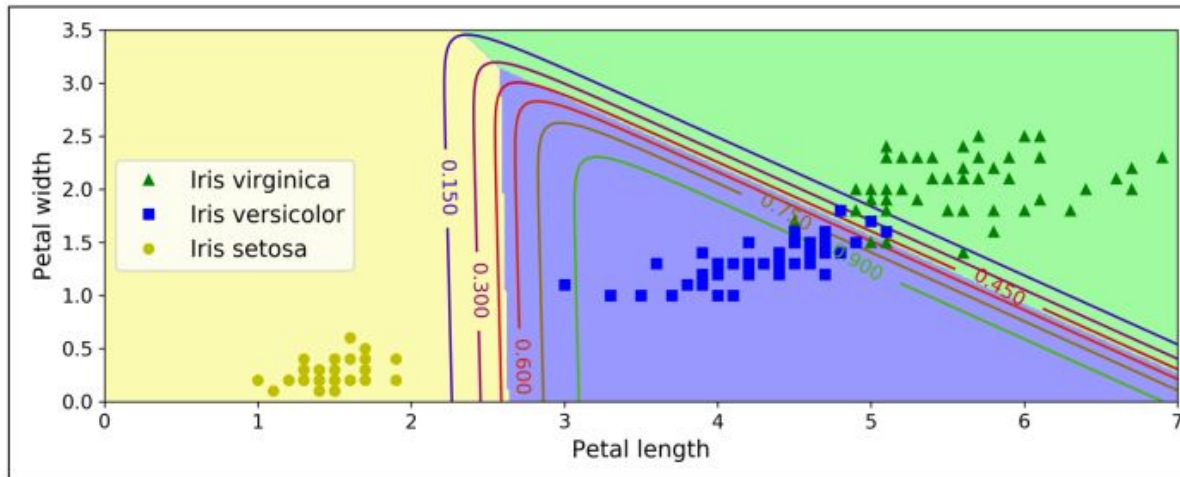


Figure 4-25. Softmax Regression decision boundaries

```
>>> softmax_reg.predict([[5, 2]])  
array([2])  
>>> softmax_reg.predict_proba([[5, 2]])  
array([[6.38014896e-07, 5.74929995e-02, 9.42506362e-01]])
```